

Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	29 September 2026
Project Name	Metro Ticket Generating System in ServiceNow
Maximum Marks	4 Marks

Functional Requirements:

The following functional requirements define the core behaviour of the proposed ServiceNow-based metro ticketing solution. They are derived from the actual project workflow: Service Catalog booking, fare logic, UI policies and catalog client scripts, Flow Designer automation, structured ticket storage, QR-based digital ticketing, request tracking, and role-based administration.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Metro Ticket Booking	• Provide a Service Catalog item for passengers to book a metro ticket. • Capture Starting From (starting_from), Going To (going_to), Number of Passengers (no_of_passengers), Type of Journey (type_of_journey: 1 Single, 2 Return) and Payment Mode. • Validate mandatory journey inputs before submission.
FR-2	Station & Journey Selection	• Present valid starting and destination stations as reference variables to the Metro Station's Details table (Vyttila, MG Road, Kaloor, Edapally, Aluva). • Prevent incomplete or invalid route selections, including the same station chosen as source and destination. • Use station order as an input to fare calculation.
FR-3	Automated Fare Calculation	• Calculate the fare from the number of stops between the selected stations, mapped to fare bands (Rs. 20 / 30 / 40 / 50). • Apply the Number of Passengers to calculate the final amount and a 2x multiplier for return journeys. • Auto-fill amount_for_single_journey or amount_including_return and display the fare during booking, before confirmation. • Read values from the form display values on the client so the fare updates instantly without database queries. • Maintain consistent fare output for repeated valid inputs.
FR-4	Request Submission & Workflow	• Create the ServiceNow request / requested item after submission. • Trigger the configured Flow Designer process. • Use request variables as inputs to downstream automation. • Create required tasks dynamically where applicable.
FR-5	Booking Data Storage	• Map catalog variables to fields in the custom Metro Database table. • Store structured booking / ticket information for operational use. • Maintain linkage between the request context and metro ticket record.
FR-6	QR Digital Ticket Generation	• Generate or prepare a QR-based digital ticket after successful processing. • Encode the required booking/ticket information. • Display the QR ticket through a Service Portal-compatible widget (HTML template with server script) / spModal flow. • Ensure the displayed ticket corresponds to the stored booking data.
FR-7	Request & Ticket Status Tracking	• Allow the passenger to view the submitted request status. • Reflect the progression of the booking workflow. • Provide clear confirmation when the ticket is ready / generated.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-8	Ticket Verification Support	• Allow station staff / authorized operations users to access the relevant ticket information. • Support QR-based validation against the booking record. • Present sufficient journey data for verification.
FR-9	Role & Access Management	• Use roles and ACLs to restrict access to protected booking data. • Separate passenger/requester access from operational/admin access. • Allow authorized administrators to configure and maintain the solution.
FR-10	UI Behaviour & Dynamic Form Logic	• Use UI Policies to show or hide the single-journey and including-return amount fields based on Type of Journey. • Use Catalog Client Scripts (onChange, UI Type All for Service Portal compatibility) on Starting From, Going To, Number of Passengers and Type of Journey for dynamic fare population and route validation. • Keep the booking experience clear and minimal.
FR-11	Workflow Monitoring & Troubleshooting	• Provide Flow Designer Execution Details for tracing failed actions and condition paths. • Support debugging of variable population, fare mismatches and automation failures through logs / execution information. • Allow administrators to verify mappings and outputs during testing.
FR-12	Notifications & Handoff Documentation	• Support configured booking/request notifications where used. • Document catalog, flow, table, ACL, testing and deployment setup. • Provide sufficient setup guidance to recreate the solution in another Personal Developer Instance.
FR-13	Application Scope & Packaging	• Build all tables, catalog items, scripts and widgets inside a custom scoped application using ServiceNow Studio. • Track changes within the application so the solution can be moved or recreated cleanly.

Non-functional Requirements:

The non-functional requirements define the quality expectations for usability, security, reliability, performance, availability and scalability. These requirements are expressed at the application design level; infrastructure-level guarantees remain dependent on the ServiceNow platform and the deployed instance.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The passenger-facing catalog form should be simple, readable and guided. Required fields, dynamic visibility of amount fields and validation should reduce confusion. The QR ticket should be easy to open and present from the portal.
NFR-2	Security	Access to booking and operational data should be governed through ServiceNow roles and ACLs. Users should only access information permitted by their role. Administrative configuration should remain restricted to authorized users.
NFR-3	Reliability	For valid inputs, the same booking data should be transferred consistently from catalog variables through Flow Designer into the Metro Database. Fare results and QR ticket data should correspond to the submitted journey information.
NFR-4	Performance	Normal booking actions should complete without unnecessary manual steps or avoidable client-side processing. Fare calculation runs on the client from on-screen values, so the amount updates instantly without database lookups. Form scripts should be kept minimal, and Flow Designer should be used for maintainable server-side workflow processing.
NFR-5	Availability	The application is expected to be accessible through the deployed ServiceNow instance whenever that instance and required platform services are available. No independent uptime guarantee is asserted by this project.

NFR No.	Non-Functional Requirement	Description
NFR-6	Scalability	The design should support adding more stations, fare mappings, journey types, ticket records and workflow rules without redesigning the full solution. The current fare logic uses a fixed station order for five stations, so a larger network would move the fare rules into a lookup table. Future payment, email/SMS and analytics functions should be attachable as extensions.
NFR-7	Maintainability	Prefer configuration and low-code components such as Service Catalog, Flow Designer, UI Policies and ACLs. Custom scripting should be limited to necessary behaviour. Setup documentation should support handoff to another administrator/developer.
NFR-8	Data Integrity	Source, destination, passenger count, fare and ticket information must remain consistent across the request and the custom Metro Database record. Validation and mapping errors should be detectable during testing.
NFR-9	Compatibility	The passenger experience should work in the intended Service Portal context, including catalog client scripts set to UI Type All. QR display should use a portal-compatible widget/spModal approach rather than depending on backend-only rendering behaviour.
NFR-10	Auditability & Troubleshooting	Administrators should be able to trace workflow execution, inspect condition paths, review stored data, and use available logs/execution details to diagnose failures.

Future / Out-of-Scope Requirements for the Current Version

- Payment gateway integration for UPI, cards or wallets (the current form only records the payment mode).
- SMS and email delivery of the completed digital ticket.
- WhatsApp-based booking or ticket delivery.
- Travel analytics dashboard and passenger-behaviour analysis.
- Additional mobility integrations such as passes or wider metro ecosystem services.

Requirement Traceability Summary

The critical end-to-end requirement path is: passenger enters journey data -> fare is calculated -> request is submitted -> Flow Designer processes the booking -> data is stored in the Metro Database -> QR ticket is displayed -> authorized station/operations users can verify the ticket. This path also defines the highest-priority testing and troubleshooting scenarios.